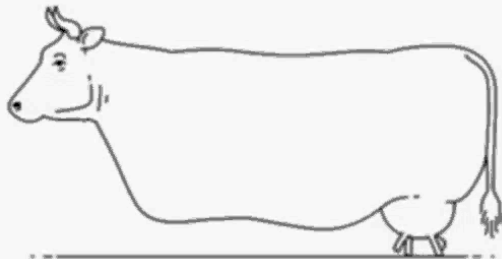


(U4284) Python程式設計

Basic

- If your code works fine don't touch it
+ my code:



Speaker: 吳淳硯



Class Contents

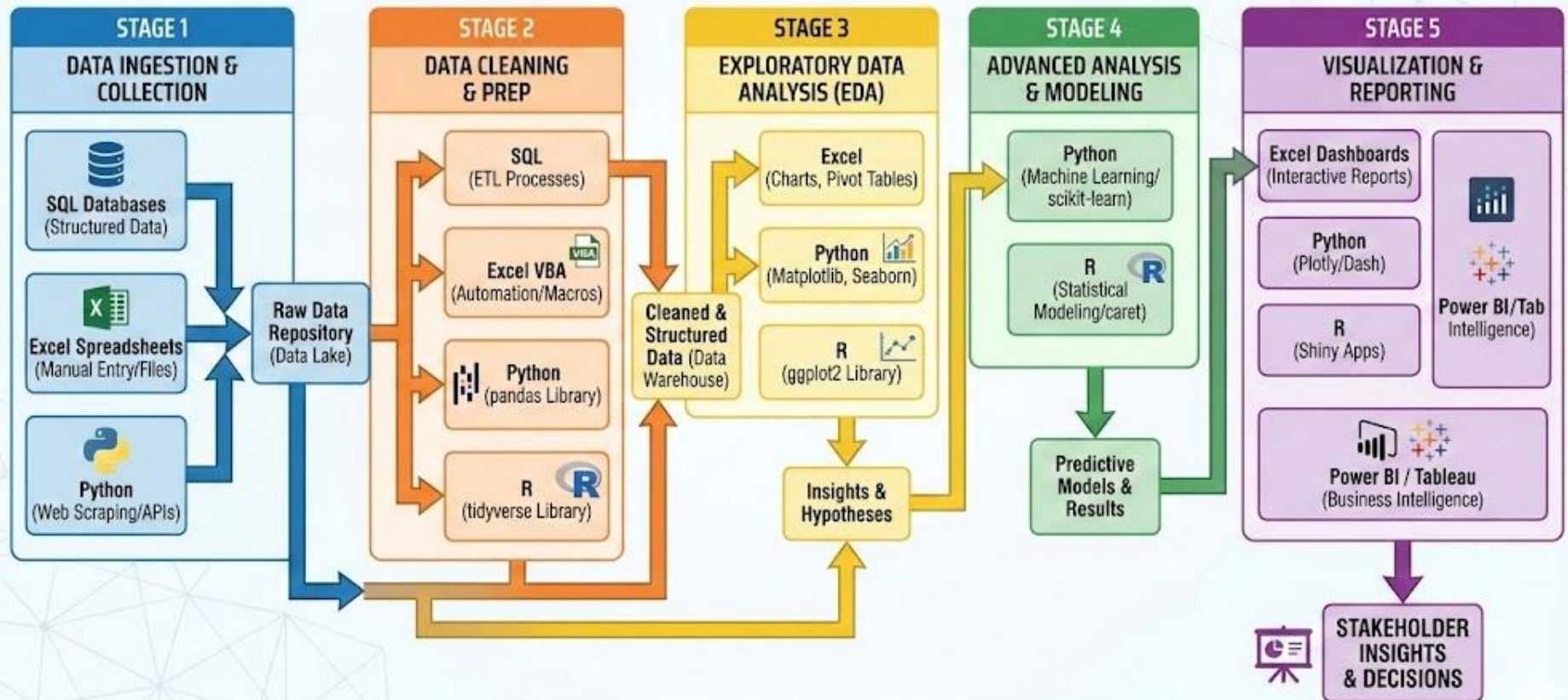
- Contents
 - 1) Basic
 - 2) Flow Control
 - 3) Function, Class
 - 4) Package Intro (numpy, scipy, pandas, seaborn, matplotlib)
 - 5) Python feat. VBA (pywin32)
 - 6) Python feat. SQL (pyodbc)
 - 7) API (fastapi)
 - 8) Linear Program (pulp)
- If time permits
 - 9) Machine Learning (scikit learn)
 - 10) Linear Model (statsmodels)
 - 11) Deep Learning (keras, tensorflow)

Evaluation & Reference

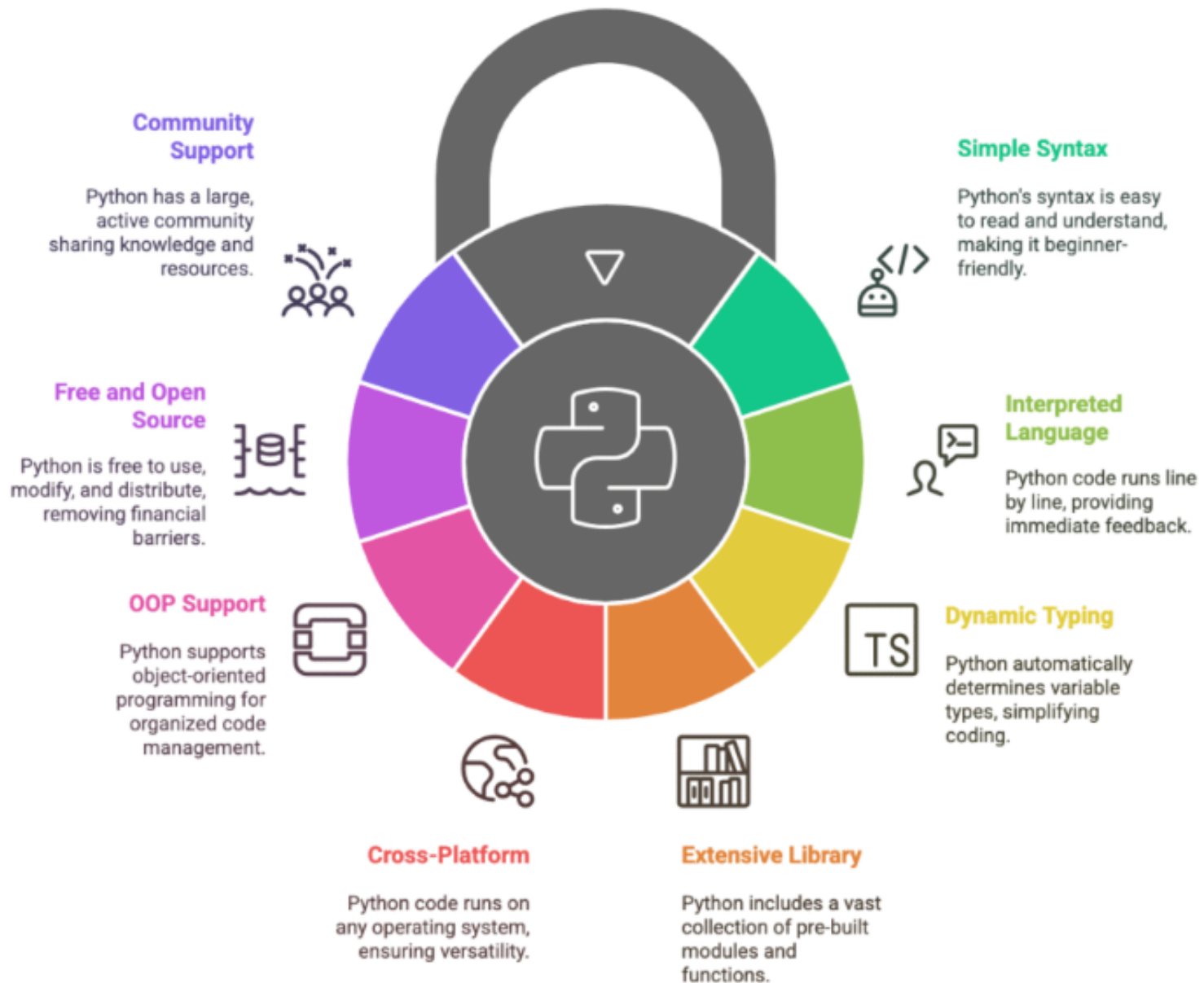
- Evaluation
 - Class Assignment ($10\% \times 4 = 40\%$)
 - Side Project or Exam ($15\% \times 2 = 30\%$)
 - Attendance ($6\% \times 5 = 30\%$)
- Teaching Methods:
 - Lecture
- Reference
 - Introduction Python – Bill Lubanovic
 - Numerical Python – Robert Johansson
 - Python Data Science Handbook – Jake VanderPlas
 - FastAPI – Bill Lubanovic
 - SQL Pocket Guide – Alice Zhao

Data Analysis Workflow

MODERN DATA ANALYSIS WORKFLOW: INTEGRATING TOOLS

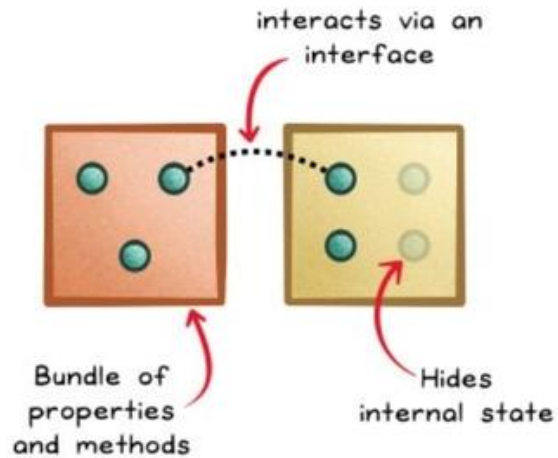


Core Features of Python

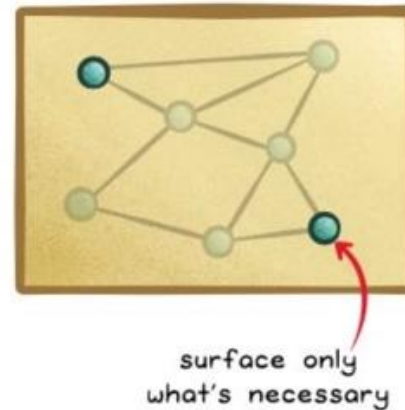


Principles of OOP (Object-Oriented Programming)

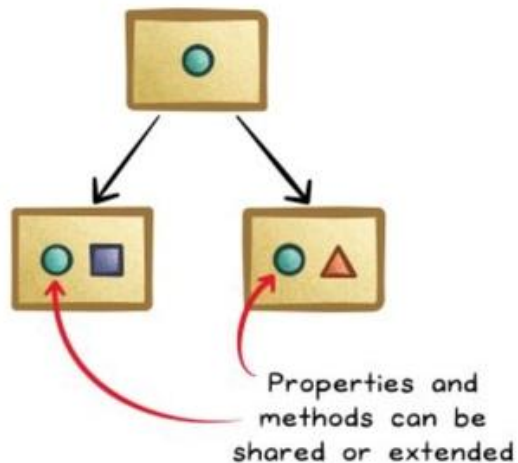
Encapsulation



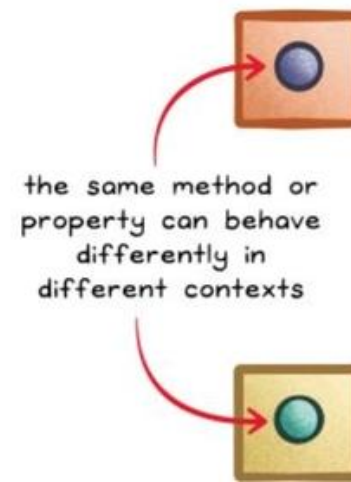
Abstraction



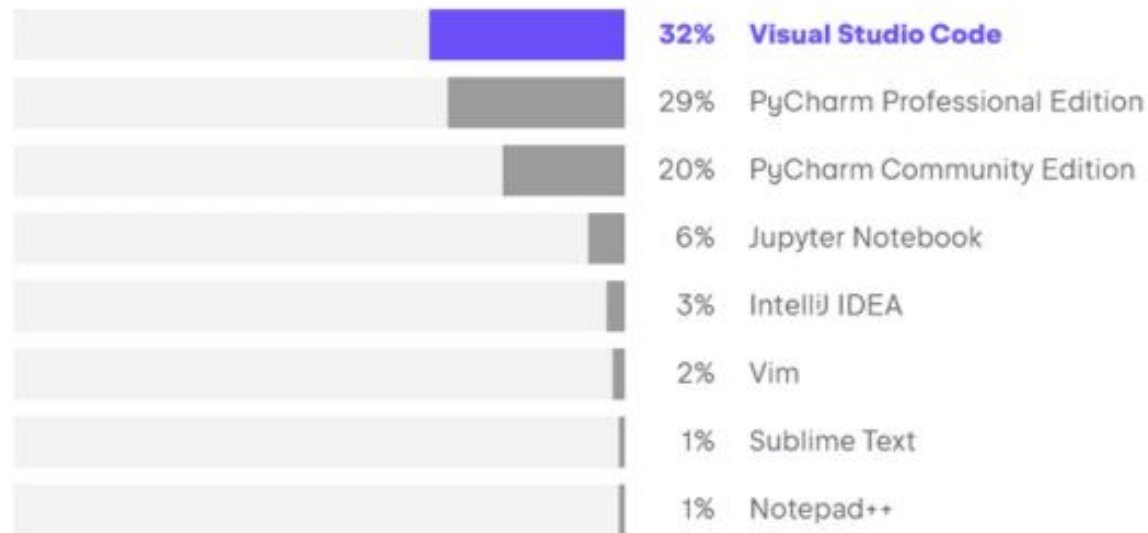
Inheritance



Polymorphism



IDE (Integrated Development Environment)



PyCharm



Visual Studio Code
(VS Code)



Jupyter
Notebook



PyDev
(Eclipse)



IDLE



Sublime
Text



Spyder



Wing



GNU/Emacs



Thonny

Cursor AI: The Next Generation Editor



Cursor IDE

An AI-powered code editor

- What is it?

An **AI-first code editor** designed to be your intelligent pair programmer.

- Why is it familiar?

If you know **VS Code**, you already know how to use Cursor—but now with deep AI integration.

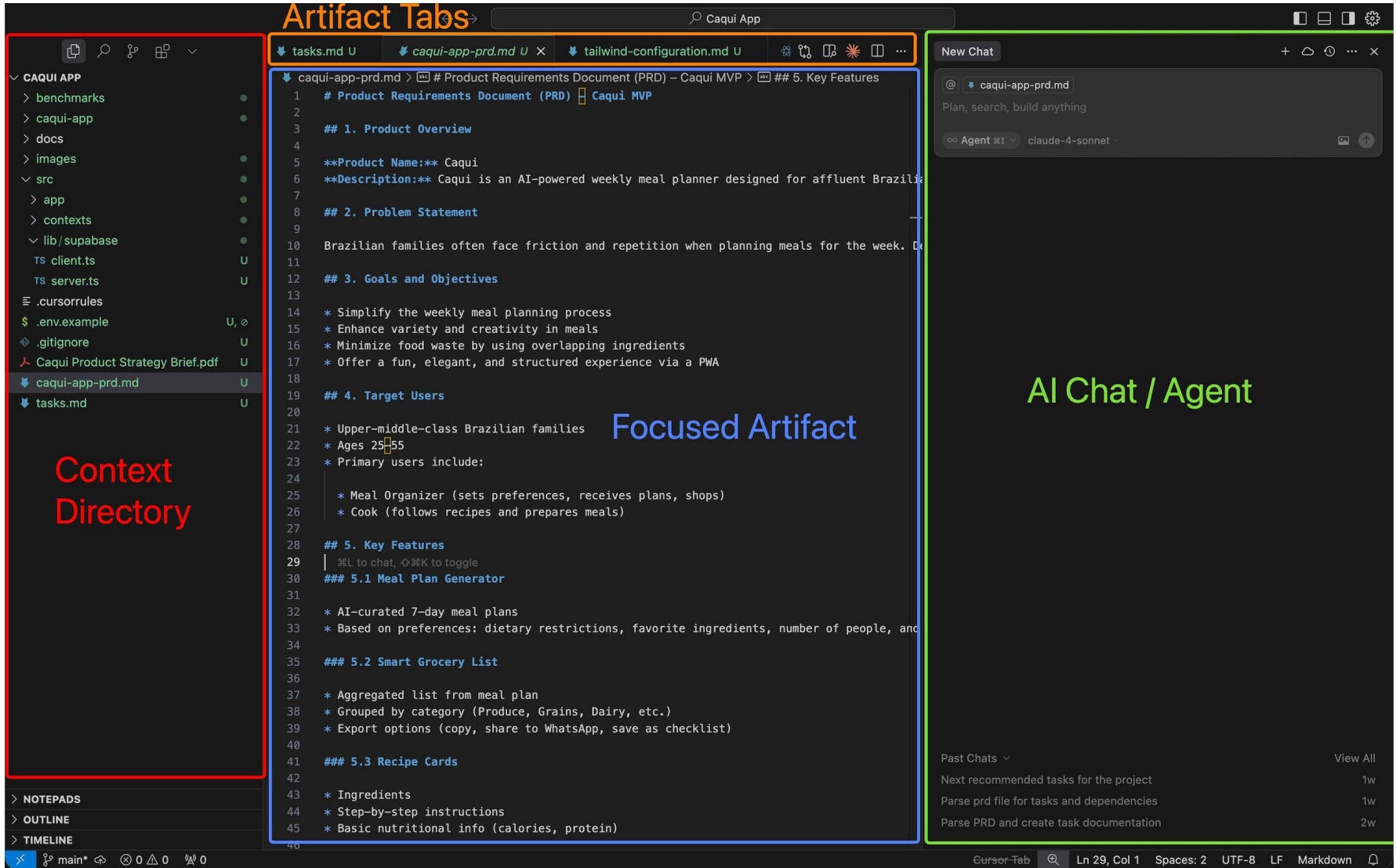
- How does it work?

Shift the focus from syntax to logic.

- **Vibe Coding** (Andrej Karpathy)
- Context-Aware Analysis

Cursor's Interface

- Three zones, each serving a distinct purpose:

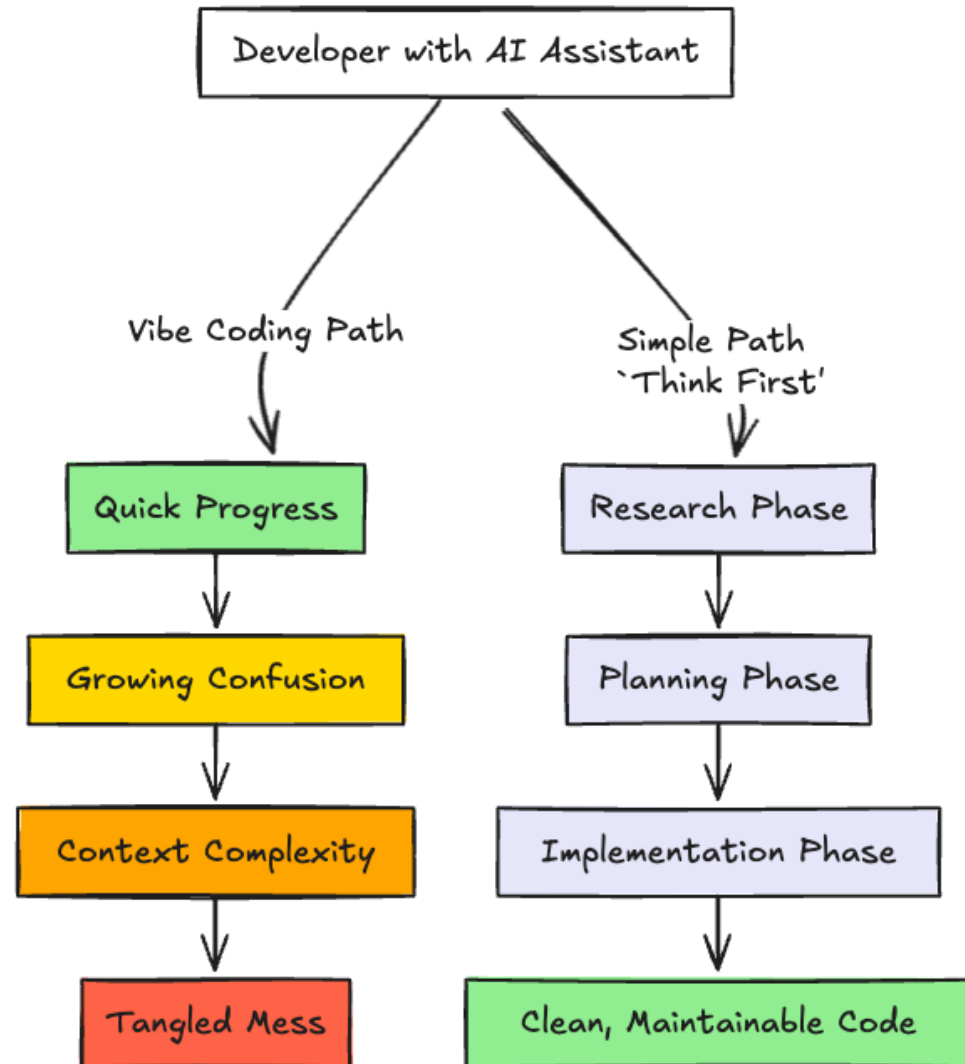


The screenshot displays the Cursor IDE interface, which is divided into three main zones:

- Context Directory (Left Panel):** A sidebar on the left showing a file tree for the 'CAQUI APP' project. It includes folders like 'benchmarks', 'caqui-app', 'docs', 'images', 'src', 'app', 'contexts', 'lib/supabase', and 'ts'. It also lists files such as 'client.ts', 'server.ts', '.cursorsrules', '.env.example', '.gitignore', 'Caqui Product Strategy Brief.pdf', 'caqui-app-prd.md', and 'tasks.md'. The 'caqui-app-prd.md' file is currently selected.
- Focused Artifact (Center Panel):** The main workspace showing the content of the selected file, 'caqui-app-prd.md'. The text is a Product Requirements Document (PRD) for 'Caqui MVP'. It includes sections for '1. Product Overview', '2. Problem Statement', '3. Goals and Objectives', '4. Target Users', and '5. Key Features'. The text is highlighted in blue.
- AI Chat / Agent (Right Panel):** A sidebar on the right showing a chat window titled 'New Chat'. It contains a message from the user: 'Plan, search, build anything'. Below the message is a response from the AI agent, 'claude-4-sonnet', which says: 'Agent: I will plan, search, build anything'. The chat window also shows a list of past chats and their timestamps.

Vibe Coding

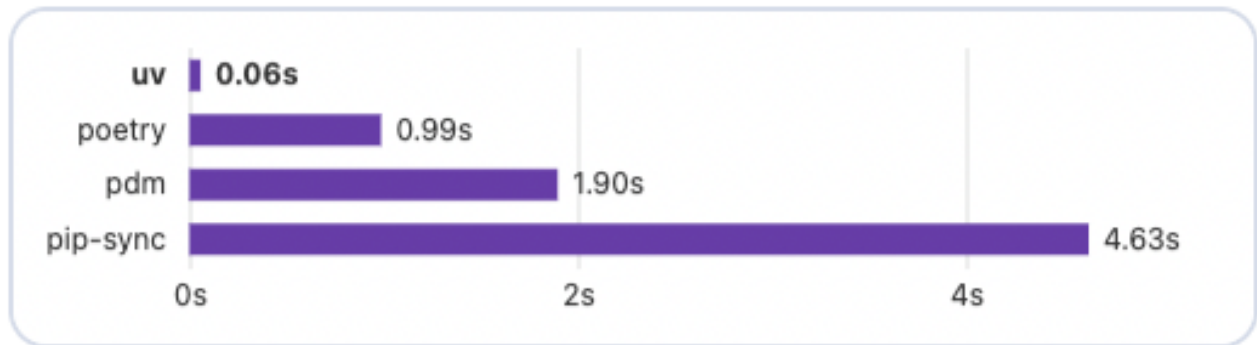
- Simple vs Easy
 - Simple:
One-fold.
 - Easy:
Adjacent.
- Vibe coding is explicitly choosing ease over simplicity
- Why This Leads to Disaster
 - The Explosion of Complexity.
 - The Illusion of Competence.
 - Technical Debt at Warp Speed.
 - The Loss of Design.



uv: The Next-Gen Python Cargo

- What is uv?
 - It is an extremely fast Python package and project manager built using **Rust**.
 - It's designed to replace and unify tools like **pip**, **pip-tools**, **pipx**, **poetry**, **pyenv**, **twine**, and **virtualenv**, offering a single, powerful Command Line Interface (CLI).

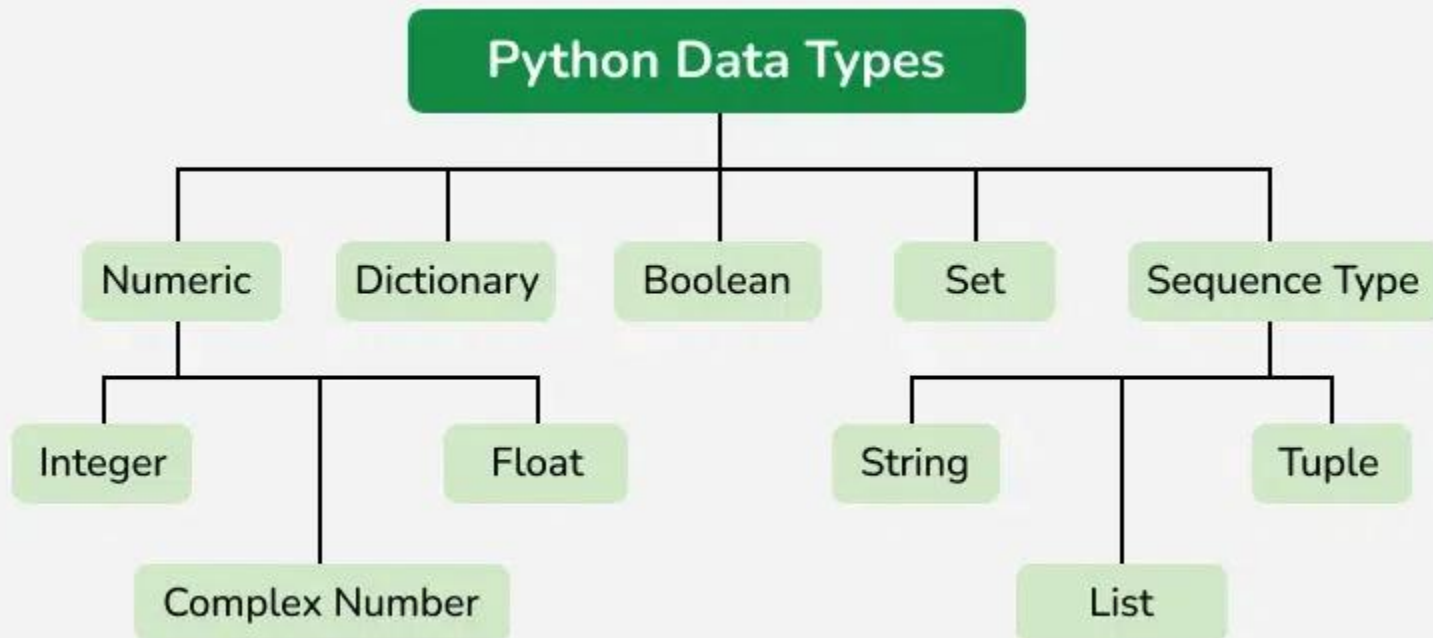
- Why uv?
 - Performance



- Python management: No more **pyenv**.
- Efficiency: Global Cache.
- Consistency: Reproducible Environments (uv.lock).

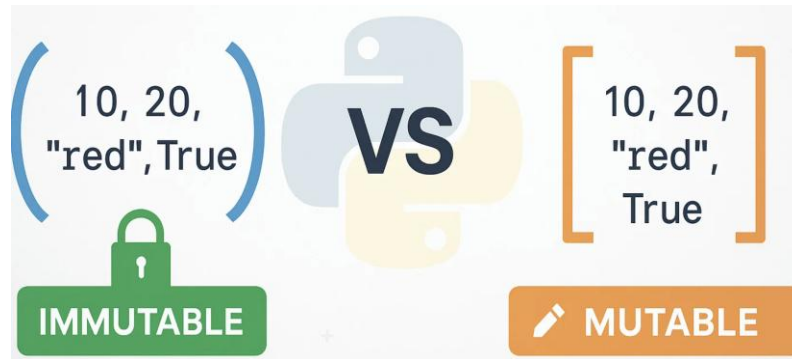
Intro

- An object is a chunk of data that contains at least the following:
 - A **type** that defines what it can do.
 - A unique **id** to distinguish it from other objects.
 - A **value** consistent with its type.
 - A **reference count** that tracks how often this object is used.



Mutability

- There are two types of objects in Python
 - Immutable: An object can't be changed after it is created.
 - Mutable: An object to be changed or modified after its creation.



- Immutable objects are quicker to access and are expensive to change because it involves the creation of a copy. Whereas mutable objects are easy to change.
- **Exception**

We know that a tuple in Python is immutable. Consider a tuple

```
tup = ([3,4,5], 15)
```

Integer are immutable so we can't change their value. But the contents of the list can change. The tuple itself isn't mutable but contains items that are mutable.

Python Variables

- Variable names can be any length and can consist of
 - Lowercase letters (a-z) & Uppercase letters (A-Z)
 - Digits (0-9)
 - Underscore (_).
- Rule of naming variables
 - A variable name cannot start with a digit.
 - Variable names are case-sensitive.
 - Avoid using Python keywords like if, else, for as variable names.

Below listed variable names are valid:

```
age = 21
_colour = "lilac"
total_score = 90
```

Below listed variables names are invalid:

```
1name = "Error" # Starts with a digit
class = 10      # 'class' is a reserved keyword
user-name = "Doe" # Contains a hyphen
```

Multi Words Variable Names

Variable names with more than one word can be difficult to read. There are several techniques you can use to make them more readable:

- Camel Case:

Each word, except the first, starts with capital letter.

```
myVariableName = "John"
```

- Pascal Case:

Each word starts with capital letter.

```
MyVariableName = "John"
```

- Snake Case:

Each word is separated by an underscore character.

```
my_variable_name = "John"
```


Python Keywords

- Identify Python Keywords
 - With Syntax Highlighting.
 - Look for Syntax Error.

```
⊗ for = 10 ...  
  
Cell In[1], line 1  
    for = 10  
      ^  
SyntaxError: invalid syntax
```

- If we attempt to use a keyword as a variable, Python will raise a **Syntax Error**.

```
help("keywords")
```

Python ▾

Here is a list of the Python keywords. Enter any keyword to get more help.

False	class	from	or
None	continue	global	pass
True	def	if	raise
and	del	import	return
as	elif	in	try
assert	else	is	while
async	except	lambda	with
await	finally	nonlocal	yield
break	for	not	

Data type

Table 2-1. Python's basic data types

Name	Type	Mutable?	Examples	Chapter
Boolean	bool	no	True, False	Chapter 3
Integer	int	no	47, 25000, 25_000	Chapter 3
Floating point	float	no	3.14, 2.7e5	Chapter 3
Complex	complex	no	3j, 5 + 9j	Chapter 22
Text string	str	no	'alas', "alack", '''a verse attack'''	Chapter 5
List	list	yes	['Winken', 'Blinken', 'Nod']	Chapter 7
Tuple	tuple	no	(2, 4, 8)	Chapter 7
Bytes	bytes	no	b'ab\xff'	Chapter 12
ByteArray	bytearray	yes	bytearray(...)	Chapter 12
Set	set	yes	set([3, 5, 7])	Chapter 8
Frozen set	frozenset	no	frozenset(['Elsa', 'Otto'])	Chapter 8
Dictionary	dict	yes	{'game': 'bingo', 'dog': 'dingo', 'drummer': 'Ringo'}	Chapter 8

Python Indentation: Logic via Space

- Indentation refers to the spaces (**PEP 8: 4 spaces**) or tabs used at the beginning of a line to define the structure of the code.
- Indentation is crucial because it indicates block levels, such as those for loops, functions, and conditions

- Why Indentation Matters?

- Syntax Enforcement.
- Readability.
- Debugging.

```
1  if True:
2      print("This is true")
3      print("Still true")
```

```
1  if True:
2  print("This will raise an error")
```

- How to Use Indentation Correctly?

- Consistent Use of Spaces or Tabs.
- No Mixing.
- Indentation Level.

```
1  for i in range(3):
2      print("Outer loop")
3      for j in range(2):
4          print("  Inner loop")
```

Python Comments

- Comments in Python are the lines in the code that are ignored by the interpreter during the execution of the program.
 - It enhance the readability of the code.
 - It can be used to identify functionality or structure the code-base.
 - It can help understanding unusual or tricky scenarios handled by the code to prevent accidental removal or changes.
 - It can be used to prevent executing any specific part of your code, while making changes or testing.

- Single Line Comments

It starts with hashtag symbol (#).

```
# sample comment  
name = "geeksforgeeks"  
print(name)
```

- Multi-Line Comments

1. Using multiple (#).
2. Using String Literals.

```
# Python program to demonstrate  
# multiline comments  
print("Multiline comments")
```

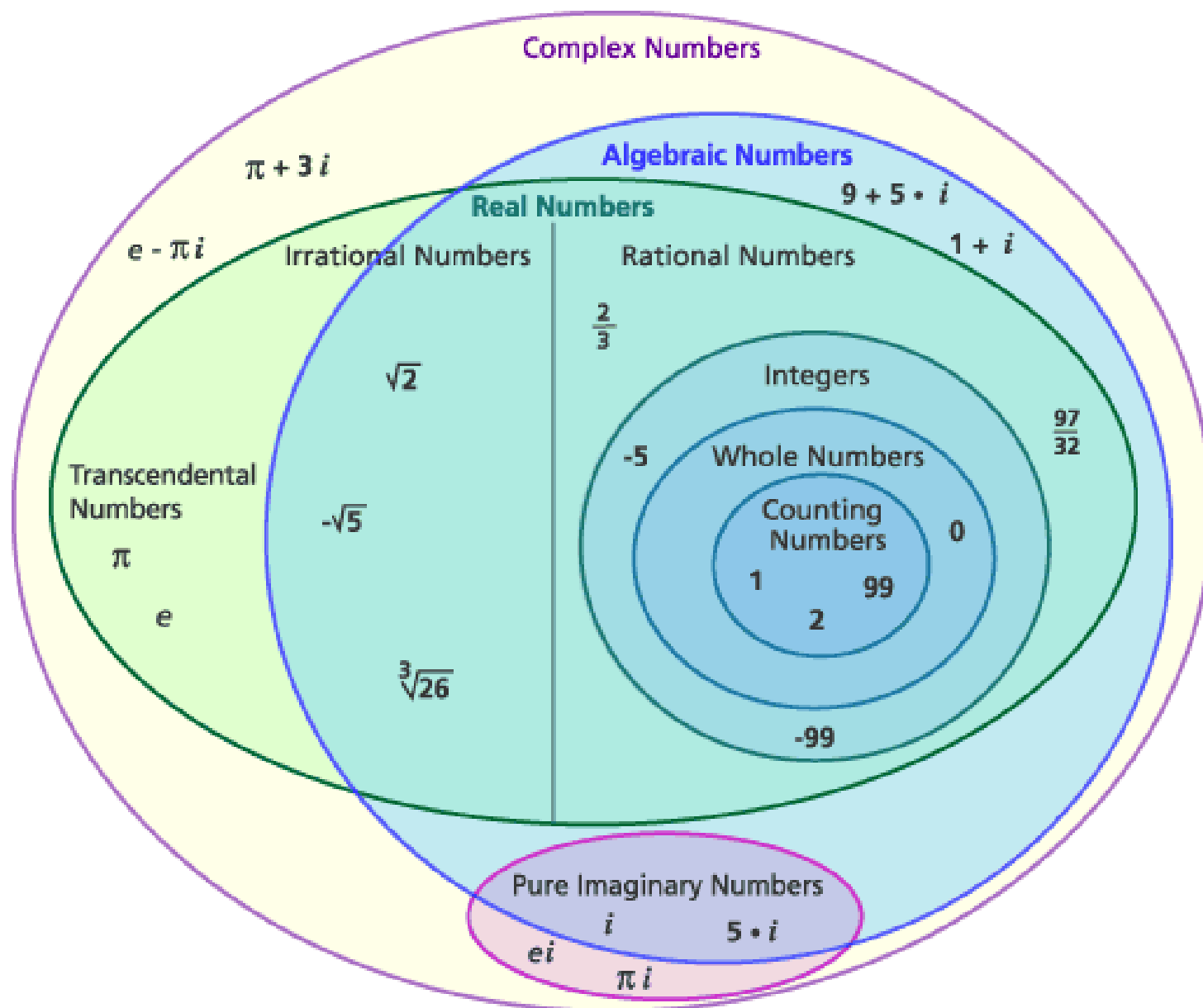
Assignment

- Computer program lines may look like equations, but their meaning is different. In Python, use = to **assign** a value to a variable.
- In math, = means **equality** of both sides, but programs it means **assignment** (assign the value on the right side to the variable on the left side).
- In programs, everything on the right side needs to have a value (this is called being **initialized**). The right side can be a literal value, or a variable that has already been assigned a value, or a combination.



We all learned in grade school arithmetic that = means *equal to*. So why do many computer languages, including Python, use = for assignment? One reason is that standard keyboards lack logical alternatives such as a left arrow key, and = didn't seem too confusing. Also, in computer programs you use assignment much more than you test for equality.

Number systems

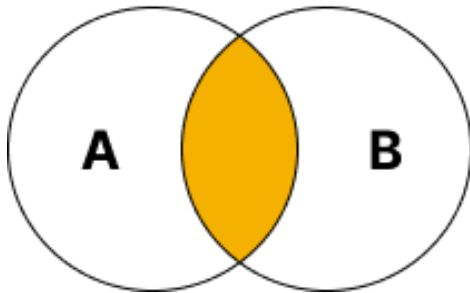


Numbers

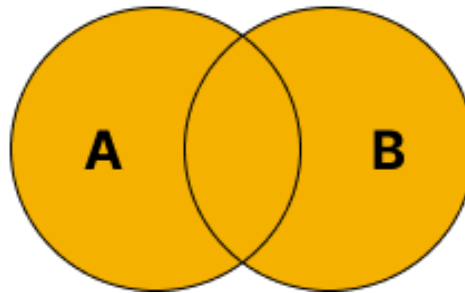
- **Booleans** – True or False.

Boolean Logic

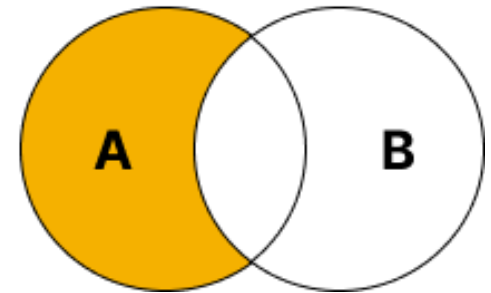
AND
both terms



OR
either term



AND NOT
excludes term



- Integers – No fractions, no decimal points.
 $\mathbb{Z} = \{\dots, -1, 0, 1, \dots\}$
- Floats – Numbers with decimal points.

Boolean algebra

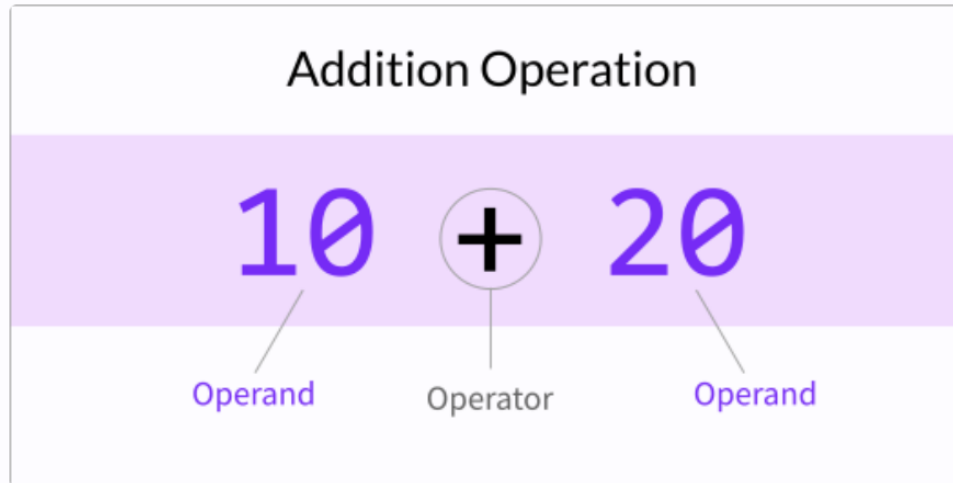
- **Boolean algebra** is a set $\mathcal{B}$ equipped with
 - Two binary operations $\vee$ (meet | and) & $\wedge$ (join | or)
 - A unary operation $\sim$ (complement | not)
 - Two elements 0 and 1 in $\mathcal{B}$
- For all elements a, b and c of $\mathcal{B}$, the following axioms hold.

$a \vee (b \vee c) = (a \vee b) \vee c$	$a \wedge (b \wedge c) = (a \wedge b) \wedge c$	associativity
$a \vee b = b \vee a$	$a \wedge b = b \wedge a$	commutativity
$a \vee (a \wedge b) = a$	$a \wedge (a \vee b) = a$	absorption
$a \vee 0 = a$	$a \wedge 1 = a$	identity
$a \vee (b \wedge c) = (a \vee b) \wedge (a \vee c)$	$a \wedge (b \vee c) = (a \wedge b) \vee (a \wedge c)$	distributivity
$a \vee \neg a = 1$	$a \wedge \neg a = 0$	complements

What are operations?

- **(Def)**

An operation can be thought of as a process or a function that's performed on given values.



- **(Def)**

An **operator** is a symbol, or keyword, that represents a given operation.

- **(Def)**

An operand is a value on which a given operation takes place.

Python Operators

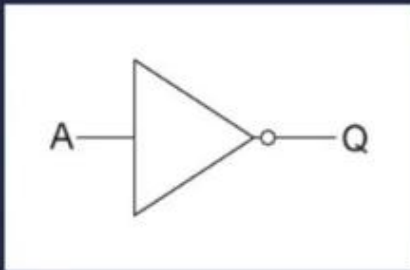
Types	Operators
Arithmetic Operators	+, -, *, /, %, **, //
Assignment Operators	+=, -=, *=, /=, %=, **=, //=
Comparison/Relational Operators	==, !=, >, <, >=, <=
Logical Operators	AND, OR, NOT
Bitwise Operators	&, , ^, ~, <<, >>
Identity Operators	is, is not
Membership Operators	in, not in

Operator Precedence

Precedence	Operator Sign	Operator Name
Highest	**	Exponentiation
	+X, -X, ~X	Unary positive, unary negative, bitwise negation
	*, /, //, %	Multiplication, division, floor, division, modulus
	+, -	Addition, subtraction
	<<, >>	Left-shift, right-shift
	&	Bitwise AND
	^	Bitwise XOR
		Bitwise OR
	==, !=, <, <=, >, >=, is, is not	Comparison, identity
	not	Boolean NOT
	and	Boolean AND
Lowest	or	Boolean OR

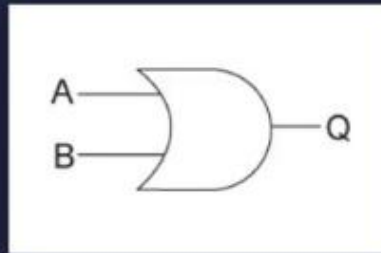
Logic Gates

- A logic gate is a circuit that implements a Boolean operation
- We draw these using symbols



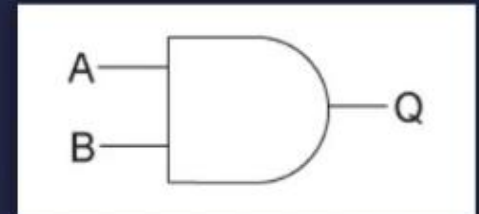
$$\text{NOT } A = Q$$

A	Q
0	1
1	0



$$A \text{ OR } B = Q$$

A	B	Q
0	0	0
0	1	1
1	0	1
1	1	1



$$A \text{ AND } B = Q$$

A	B	Q
0	0	0
0	1	0
1	0	0
1	1	1

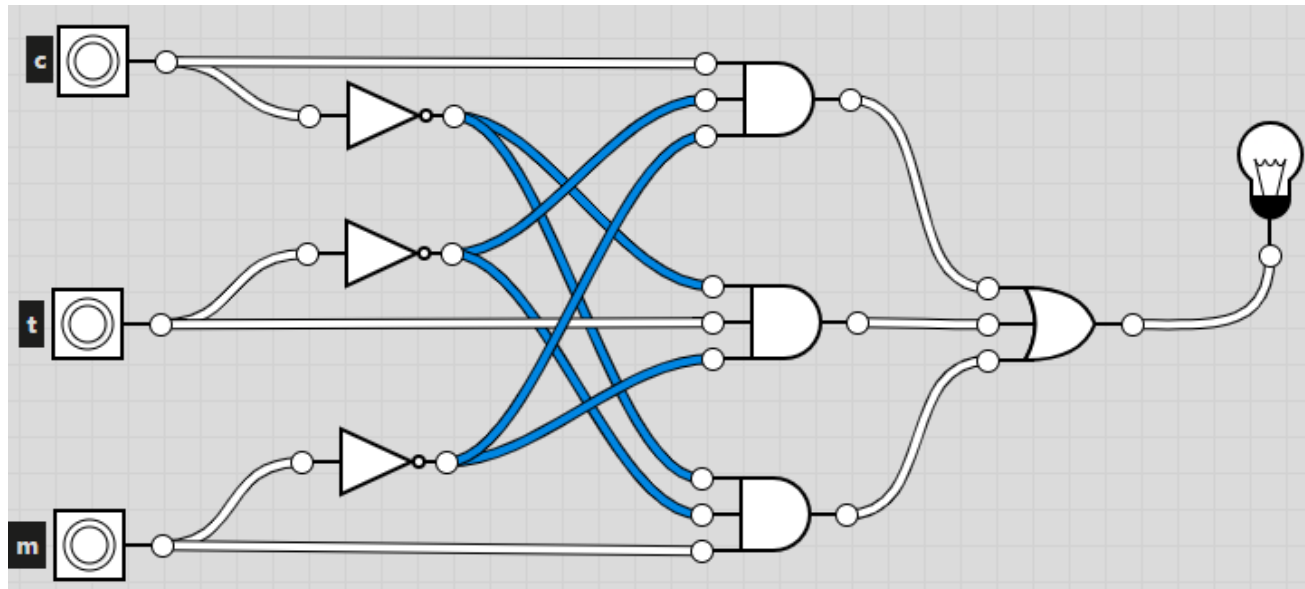
Coffee, Tea or Milk?

- Input

- c = coffee button (1 = pushed, 0 = not pushed)
- t = tea button (1 = pushed, 0 = not pushed)
- m = milk button (1 = pushed, 0 = not pushed)

- Output

- x = choice verifier (1 = acceptable input / deliver the choice,
0 = unacceptable input / light an error light)
- $$= c(\sim t)(\sim m) + (\sim c)t(\sim m) + (\sim c)(\sim t)m$$



c	t	m	x
1	1	1	0
1	1	0	0
1	0	1	0
1	0	0	1
0	1	1	0
0	1	0	1
0	0	1	1
0	0	0	0

Exercise – Two Floor Elevator

There is a two-floor elevator:

deciding when to move to the other floor.

- Input
 - f = 1st floor button (1 = pushed, 0 = not pushed)
 - s = 2nd floor button (1 = pushed, 0 = not pushed)
 - p = present floor indicator (1 = 1st floor, 0 = 2nd floor)
- Output
 - m = move function (1 = move or change floor, 0 = stay)

☺ Question ?

- a. Write the Boolean expression.
- b. Draw the circuit.
- c. Construct the truth table listing all possibilities

Integers

$\mathbb{Z}$ under addition $(\mathbb{Z}, +)$
is a abelian group

	Addition	Multiplication
Closure:	$a + b$ is an integer	$a \times b$ is an integer
Associativity:	$a + (b + c) = (a + b) + c$	$a \times (b \times c) = (a \times b) \times c$
Commutativity:	$a + b = b + a$	$a \times b = b \times a$
Existence of an identity element:	$a + 0 = a$	$a \times 1 = a$
Existence of inverse elements:	$a + (-a) = 0$	The only invertible integers (called units) are -1 and 1 .
Distributivity:	$a \times (b + c) = (a \times b) + (a \times c)$ and $(a + b) \times c = (a \times c) + (b \times c)$	
No zero divisors:		If $a \times b = 0$, then $a = 0$ or $b = 0$ (or both)

Positional Number System

- In positional number systems, a number is represented by a string of digits where position of each digit is associated with a weight. In general, is expressed as

$$d_{m-1}d_{m-2} \cdots d_1d_0.d_{-1}d_{-2} \cdots d_{-n}$$

where d_{m-1} is referred to as the **most significant digit (MSD)** and d_{-n} as the **least significant digit (LSD)**.

- Integers are assumed to be **decimal** (base 10) unless you use prefix to specify another base.
- In Python, you can express literal integers in three bases beside decimal with these integer prefixes:
 - 0b or 0B for **binary** (base 2)
 - 0o or 0O for **octal** (base 8)
 - 0x or 0X for **hex** (base 16)

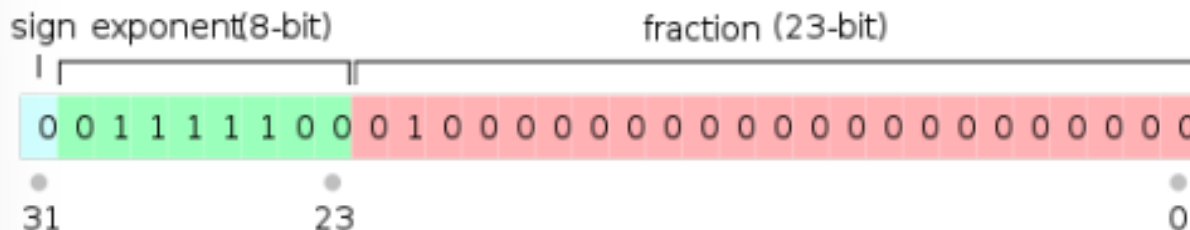
Floating point representation

- A floating point number in base (b)

$$x = (-1)^s \times m \times b^e$$

consists of below components:

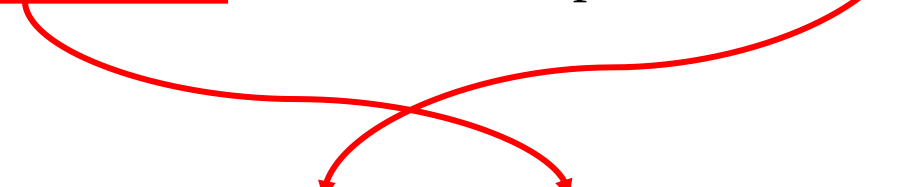
- Sign (s) and Base (b)
 - Mantissa ($m = (1.f)_b$)
 - Exponent ($e = E - \text{bias} = E - [2^{k-1} - 1]$)
- IEEE standard formats :
 - single : 23 bit fraction, 8 bit exponent
 - double : 52 bit fraction, 11 bit exponent



$$\begin{aligned}
 (-1)^s(1.f)_2 2^e &= (-1)^0(1.01)_2 2^{(01111100)_2 - [2^{8-1} - 1]} \\
 &= (1 + 1 \times 2^{-2}) 2^{124 - 127} \\
 &= (1.25) 2^{-3} = 0.15625
 \end{aligned}$$

What is Regular Expressions(RE)?

- String containing a combination of **normal characters** and special **metacharacters** that describes patterns to find text or position within a text.



r'st\d\s\w{3,10}'

The diagram shows two red boxes: one around 'normal characters' and one around 'metacharacters' in the text above. Red arrows point from these boxes to the components of the regular expression 'r'st\d\s\w{3,10}'. The 'r' is highlighted in pink. 'st' is enclosed in a red box, as is '\d\s\w'. The '{3,10}' is also enclosed in a red box.

- The re module
 - re.findall – return a list of all non-overlapping matches, if any.
 - re.split – split **source** at matches with **pattern** and return a list of the string pieces.
 - re.sub – take another **replacement** argument, and changes all parts of **source** that are matched by **pattern** to **replacement**.
 - re.search – return the first match, if any.
 - re.match – return exact beginning match, if any.

Metacharacters

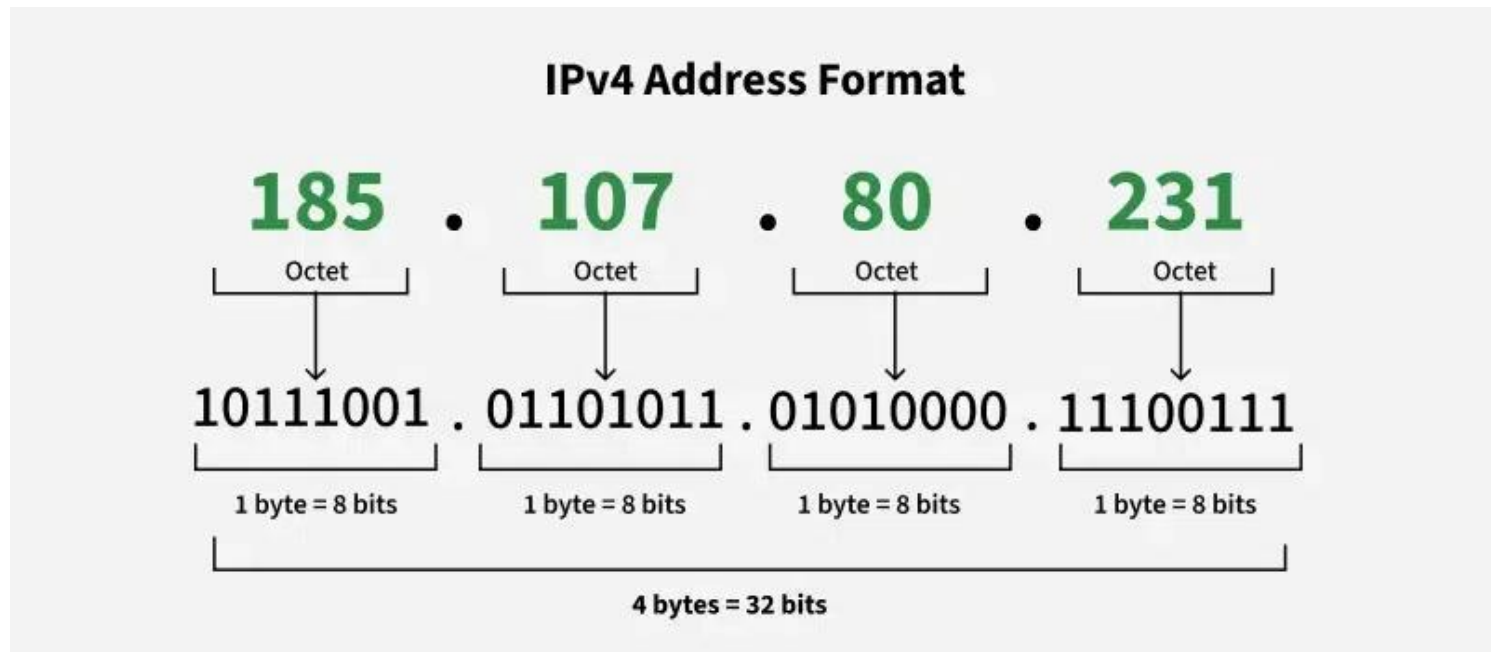
Character	Description	Example
[]	A set of characters	[a-m]
\	Signals a special sequence	\d
.	Any character	he..o
^	Starts with	^hello
\$	Ends with	world\$
*	0 or more occurrences	aix*
+	1 or more occurrences	aix+
?	0 or 1 occurrences	aix?
{m,n}	Exactly the specified No. of occurrences	ak{2},ak{3,6}
	Either or	call put
()	Capture and group	

Special Sequences

Character	Description
\A	Returns a match if the specified characters are at the start of the string
\b	Returns a match where the specified characters are at the start or at the end of a word
\B	Returns a match where the specified characters are present, but not at the start (or at the end) of a word
\d	Returns a match where the string contains digits
\D	Returns a match where the string does not contain digits
\s	Returns a match where the string contains a white space character
\S	Returns a match where the string does not contain a white space character
\w	Returns a match where the string contains any word characters
\W	Returns a match where the string does not contain any word characters
\Z	Returns a match if the specified characters are at the end of the string

Exercise – IP v4 & Mobile Number

- What is an IPv4 Address?
 - IPv4 address is a 32-bit numeric address written as 4 decimal numbers (called octets) separated by periods
- ☺ Write the regular expression of detect IP Address.



- Taiwan Mobile phone numbers
 - begin in three digits ranging 090~098 with a total length of 10 digits.
- ☺ Write the regular expression of detect Taiwan Mobile number.

Tuples and Lists

- The elements can be of different types. In fact, each element can be any **Python object**.
- Why does Python contain both lists and tuples?
 - **Mutability**

<u>TUPLES</u>		<u>LISTS</u>
The items are surrounded in paranthesis ().	Syntax	The items are surrounded in square brackets [].
Tuples are immutable in nature.	Mutability	Lists are mutable in nature.
There are 33 available methods on tuples.	Methods	There are 46 available methods on lists.
In dictionary, we can create keys using tuples.	Usability	In dictionary, we can't use lists as keys.

Dictionaries & Sets

- A **dictionary** is similar to a list, but the order of items does not matter, and they are not selected by an offset such as 0 or 1. Instead, you specify a unique **key** to associate with each **value**.
 - Create with **{}** or **dict()**
 - The key can actually be any of Python's immutable types.
 - Copy everything with **deepcopy()**
 - Dictionary comprehension

```
{key_expression : value_expression for expression in iterable}
```

- A **set** is like a dictionary with its values thrown away, leaving only the keys.
 - Create with **{}** or **set()**
 - Create an Immutable set with **frozenset()**
 - Set comprehension

```
{ expression for expression in iterable }
```

Summary

	Lists	Tuples	Sets	Dictionaries
Ordering	Ordered	Ordered	Unordered	Ordered Unordered before Python 3.7
Indexing	Indexed	Indexed	Not Indexed	Keyed
Mutability	Mutable	Immutable	Mutable Only Adding and Removing	Mutable
Duplicates Allowed	Yes	Yes	No	Yes Only in values and not in keys
Types Allowed	Mutable and Immutable	Mutable and Immutable	Only Immutable	Only Immutable In keys

Generator

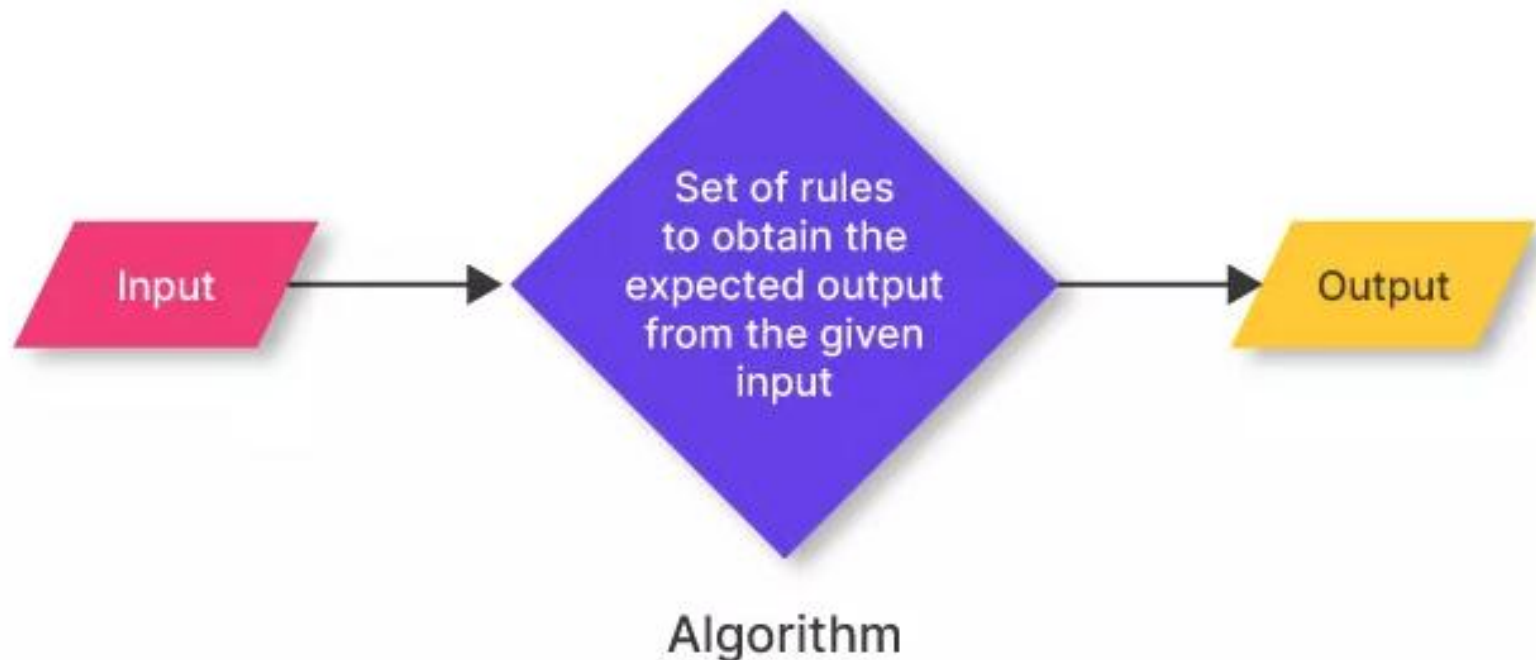
- A generator is a Python sequence creation object. With it, you can iterate through potentially huge sequences without creating and storing the entire sequence in memory at once.
- Every time you iterate through a generator, it keeps track of where it was the last time it was called and returns the next value.
- It is a normal function, but it returns its value with a **yield** statement rather than return.
- Generator Comprehension

❗ Definition:

The syntax `(<expression> for <var> in <iterable> [if <condition>])` specifies the general form for a **generator comprehension**. This produces a generator, whose instructions for generating its members are provided within the parenthetical statement.

What is an Algorithm?

- A finite set of instructions that accomplishes a particular task. In addition all algorithms must satisfy the following criteria:
 - Input (Zero or more inputs)
 - Output (At least one output)
 - Finiteness (N number of steps)
 - Definiteness (Clear algorithm step)
 - Effectiveness (A carried out step)



What is a good algorithm?

- A good algorithm implemented on a slow computer may perform much better than a bad algorithm implemented on a fast computer.
- What makes an algorithm good?
 - Good **time complexity** (maybe **space complexity**)
 - Better than any other algorithm
 - Easy to understand
- How could we measure how much work an algorithm does?
 - Code it and time it. Issues?
 - Count how many instructions it does before implementing it
 - Computer scientists count basic operations, and use a rough measure of this:

asymptotic notation

Performance Measurement

- Complexity

- Space complexity:
the amount of **memory** it needs to run to completion.
- Time complexity:
the amount of **computer (CPU) time** it needs to run to completion.

- [TSP problem]

Given a list of cities and the distances between each pair of cities, what is the shortest possible route that visits each city exactly once and returns to the origin city ?

Let n = No. of cities, number of possible solution is

$$(n - 1) \times \cdots \times 2 \times 1 = (n - 1)!$$

$(n - 1)!$ Grows very quickly as n grows

$$n = 3 \Rightarrow (n - 1)! = 2$$

$$n = 10 \Rightarrow (n - 1)! = 3628800$$

assume computer checks 3.15×10^{13} solutions every year,

$$n = 26 \approx 5 \times 10^{11} \text{ years}$$

Asymptotic notations

- [Def - Big O] (**worst case enumeration**)

$$f(n) = O(g(n))$$

if $\exists M, N$ such that

$$|f(n)| \leq M|g(n)| \quad \forall n \geq N$$

Intuitively, this means that f does not grow faster than g .

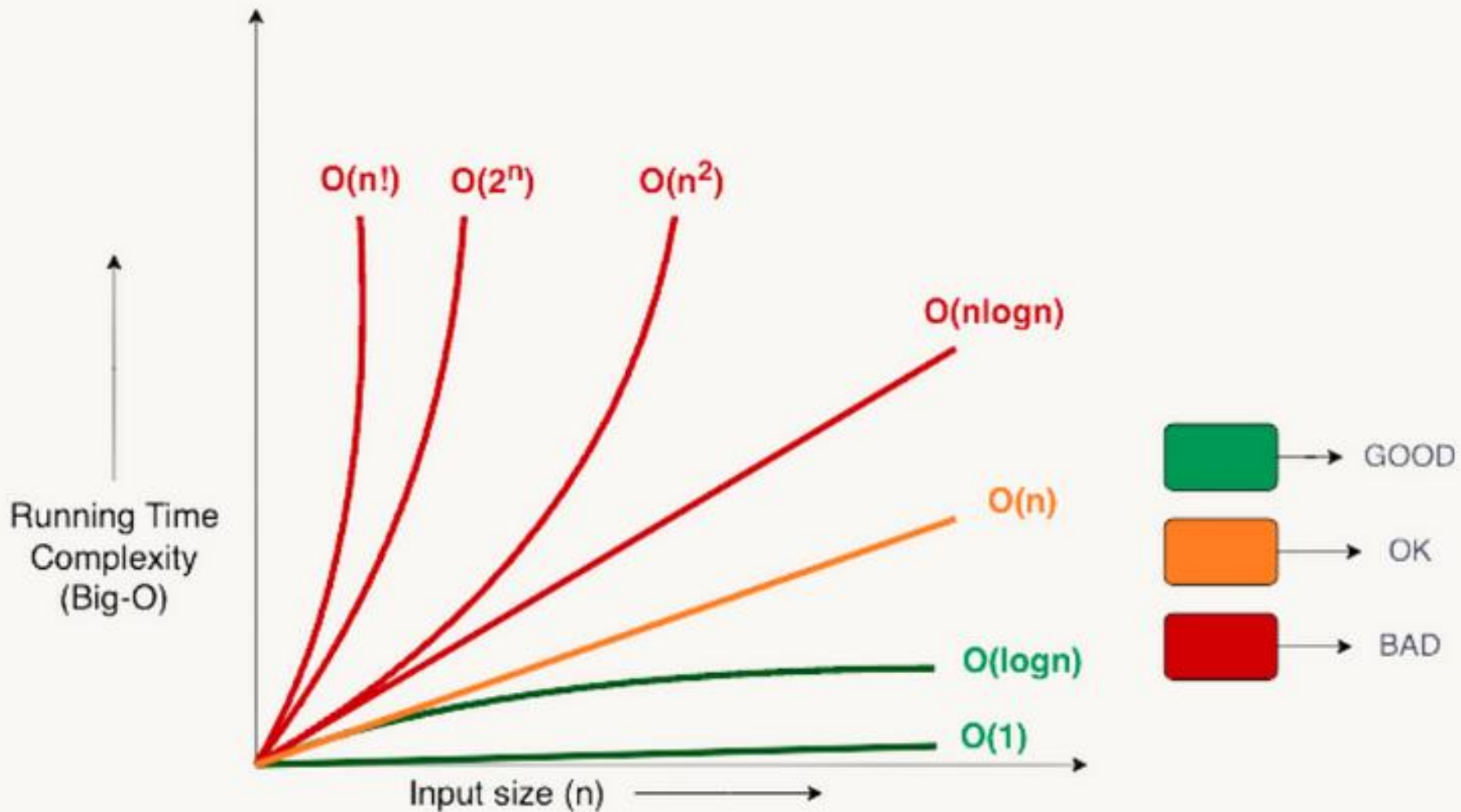
- Classes of functions that are commonly encountered when analyzing algorithms.

Notation	Meaning
$O(\log n)$	Logarithm
$O(n)$	Linear
$O(n \log n)$	Linearithmic
$O(n^2)$	Quadratic
$O(2^n)$	Exponential
$O(n!)$	factorial

Polynomial Time

Non-Polynomial Time

Big O Complexity



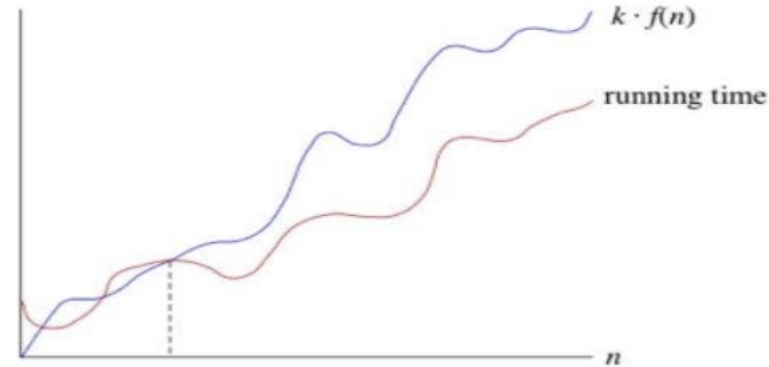
Asymptotic notations

- [Def - **Omega**] (Lower bound Enumeration)

$$f(n) = \Omega(g(n))$$

if $\exists m, N$ such that

$$|f(n)| \geq mg(n) \quad \forall n \geq N$$

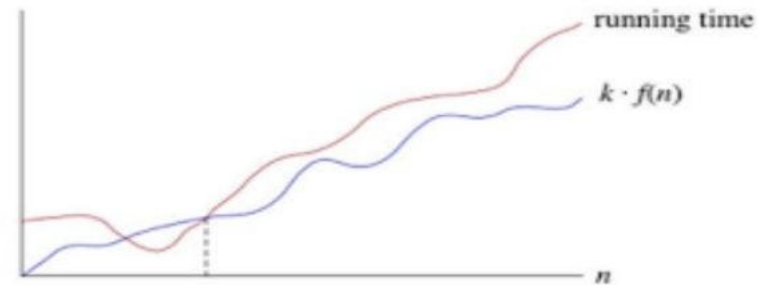


- [Def - **Theta**] (Optimal Enumeration)

$$f(n) = \Theta(g(n))$$

if $\exists m, M, N$ such that

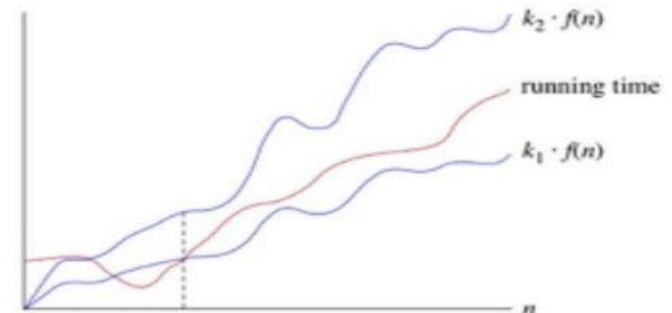
$$mg(n) \leq f(n) \leq Mg(n) \quad \forall n \geq N$$



- Equivalence

$$f(n) = \Theta(g(n))$$

$$\Leftrightarrow f(n) = O(g(n)) \text{ and } \Omega(g(n))$$



Space Complexity

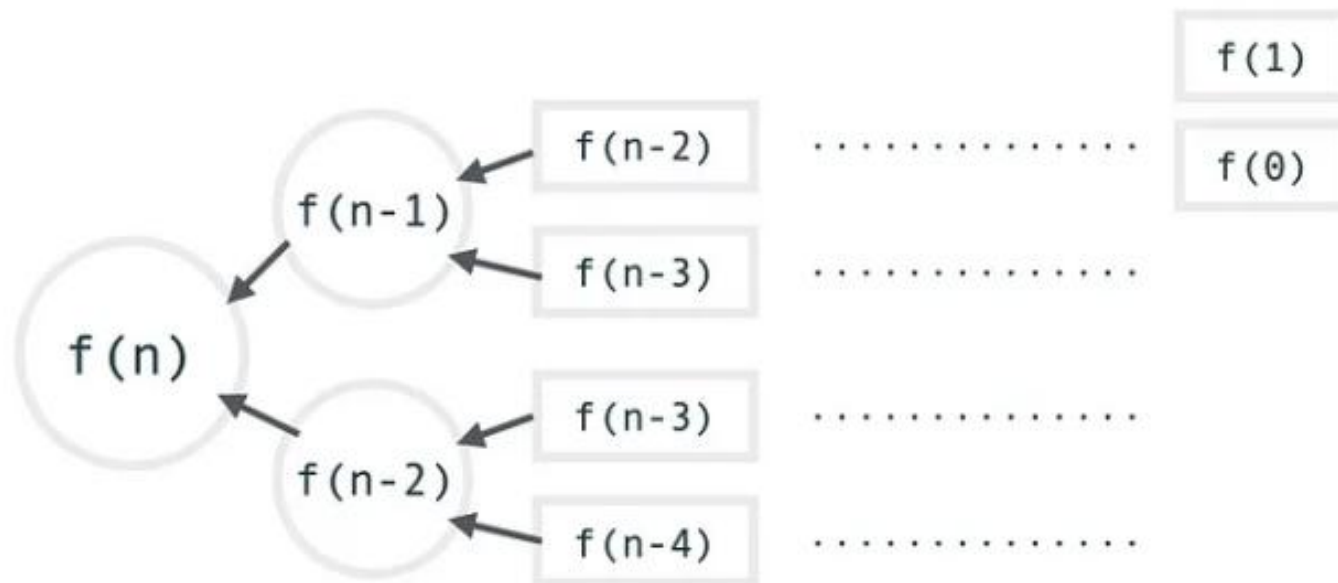
- Recall Fibonacci number

$$f(n) = f(n-1) + f(n-2)$$

$$f(0) = f(1) = 1$$

- Space Complexity is $O(N)$

- As in recursion, the space required is proportional to the **maximum depth** of the recursion tree, and it is N .



Measure the difficulty of the problems

Class	Description
P	problems that can be solved in $O(p(n))$
NP	problems that can be verified in $O(p(n))$
NP-Hard	problems that are at least as hard as all NP problems
NP-Complete	problems in both NP and NP-Hard

Polynomial
Time

TSP problem belongs
to the NP-Complete

